Fiacchi, Santini, Szczurek - HM 7

Aleksandra Szczurek, Lucia Santini, Giada Fiacchi

2025-12-03

```r
#generate using the inverse transformation method
n <- 10^5
u <- runif(n)
lambda <- 1

#inverse CDF
x_inv <- -(1/lambda) * log(u)

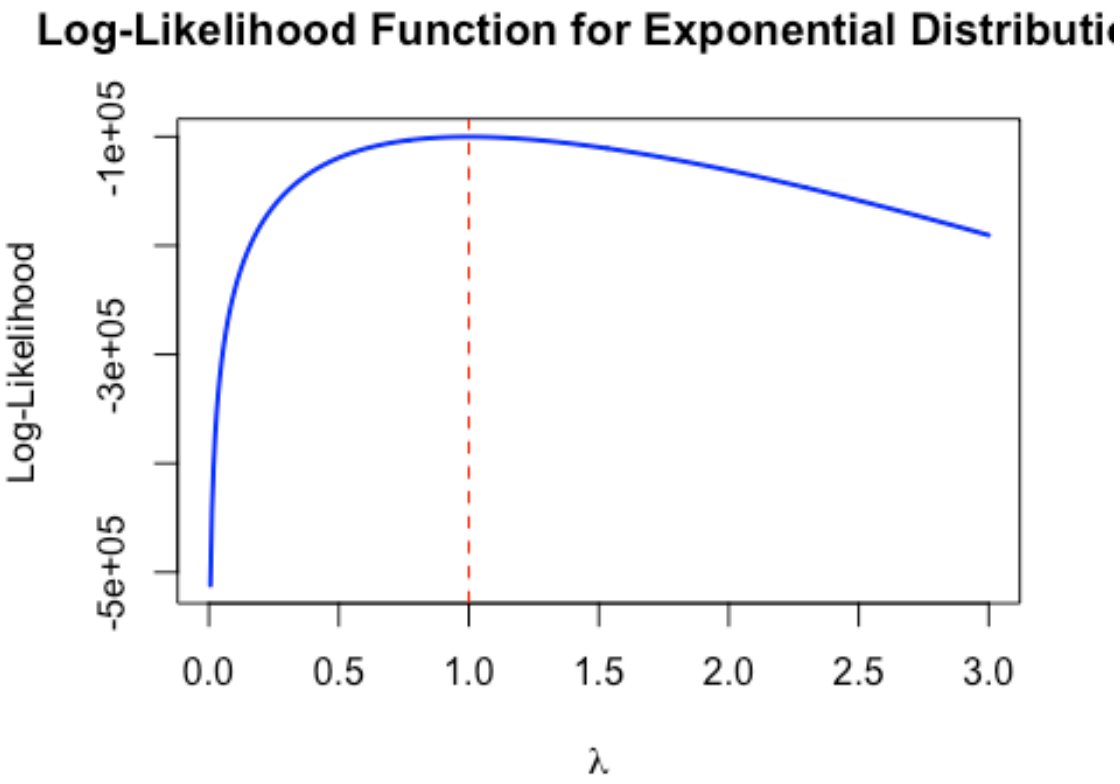
#calculate the sum of the simulated data
sum_x <- sum(x_inv)

#define the log-likelihood function (l(lambda | x))
log_likelihood <- function(lambda, n, sum_x) {
  if (lambda <= 0) {
    return(-Inf)
  }
  return(n * log(lambda) - lambda * sum_x)
}

#define a range of lambda values
lambda_values <- seq(0, 3, length.out = 500)

#calculate the log-likelihood for each lambda value
ll_values <- sapply(lambda_values, log_likelihood, n, sum_x)

#plot
plot(lambda_values, ll_values, type = 'l', col = 'blue', lwd = 2,
     main = "Log-Likelihood Function for Exponential Distribution",
     xlab = expression(lambda),
     ylab = "Log-Likelihood")
abline(v = lambda, col = 'red', lty = 2)
```



The data were generated using the inverse transform method with a very large sample size (N = 100,000), which ensures results close to the theoretical mean. They come from an exponential distribution with a true rate parameter of λ = 1. The log-likelihood plot shows a concave shape, indicating that there is a single maximum likelihood estimate. The peak of the curve appears close to λ = 1, as expected.

```r
#negative Log-Likelihood (NLL) function
nll <- function(lambda, n, sum_x) {
  if (lambda <= 0) {
    return(Inf)
  }
  return(-(n * log(lambda) - lambda * sum_x))
}

#now we can maximize log-likelihood (Minimize NLL) using optim
#select a starting value for the parameter
start_value <- 0.2

#perform using optim
mle_results_opt <- optim(
  par = start_value,
  fn = nll,
  n = n,
  sum_x = sum_x,
```

```r
  method = "Nelder-Mead",
  hessian = TRUE
); mle_results_opt
```

```
## $par
## [1] 0.9990625
##
## $value
## [1] 100106.4
##
## $counts
## function gradient
##       32       NA
##
## $convergence
## [1] 0
##
## $message
## NULL
##
## $hessian
##        [,1]
## [1,] 100188
```

```r
# The Hessian matrix (H) is returned by optim() if hessian=TRUE
H <- mle_results_opt$hessian

# The estimated variance is the inverse of the Hessian (for a single parameter)
estimated_variance_optim <- 1 / H[1, 1]

#standard error
se_mle_optim <- sqrt(estimated_variance_optim); se_mle_optim
```

```
## [1] 0.00315931
```

```r
#table for comparison
comparison_data <- data.frame(
  Parameter = c("True lambda", "MLE lambda"),
  Value = c(lambda, mle_results_opt$par),
  Difference = c("-", round(mle_results_opt$par - lambda, 4)),
  row.names = NULL
); comparison_data
```

```
##     Parameter     Value Difference
## 1 True lambda 1.0000000          -
## 2  MLE lambda 0.9990625     -9e-04
```

The maximum likelihood estimate for λ is found by minimizing the negative log-likelihood using the optim function. The estimated value is very close to the true parameter λ = 1, showing that the estimator is consistent when the sample size is large. The standard error is obtained from the

inverse of the Hessian matrix evaluated at the estimate. Because the sample size is large, this standard error is very small, indicating that the estimate of λ is highly precise.

```
#With n = 10^5, the MLE is very close to the true value, demonstrating the
consistency property of ML estimators.

#BOOTSTRAP (non parametric)
n <- length(x_inv)
B <- 999
MLE_boot <- matrix(NA, B, 1)
colnames(MLE_boot) <- c("lambda")

for(b in 1:B){
  x_boot <- sample(x_inv, n, replace = TRUE)
  lambda_boot <- 1 / mean(x_boot)
  MLE_boot[b, 1] <- lambda_boot
}

#table
comparison_data2 <- data.frame(
  Parameter = c("True lambda", "MLE lambda", "Bootstrap lambda"),
  Value = c(lambda, mle_results_opt$par, lambda_boot),
  row.names = NULL
); comparison_data2

##          Parameter     Value
## 1      True lambda 1.0000000
## 2       MLE lambda 0.9990625
## 3 Bootstrap lambda 0.9920880

#standard error bootstrap
SE_bootstrap <- apply(MLE_boot, 2, sd)

#table
comparison_data3 <- data.frame(
  SE_parameter = c("MLE", "Bootstrap"),
  Value = c(se_mle_optim, SE_bootstrap),
  row.names = NULL
); comparison_data3

##   SE_parameter       Value
## 1          MLE 0.003159310
## 2    Bootstrap 0.003234932
```

The goal of the bootstrap method is to estimate the sampling distribution of the maximum likelihood estimator and its standard error without relying on the analytical Hessian calculation. To do this, 999 estimates were obtained by repeatedly resampling the original data with replacement. The standard deviation of these bootstrap estimates gives the bootstrap standard error. The bootstrap and analytical standard errors should be very close, and this similarity confirms that the analytical approximation works well when the sample size is large.